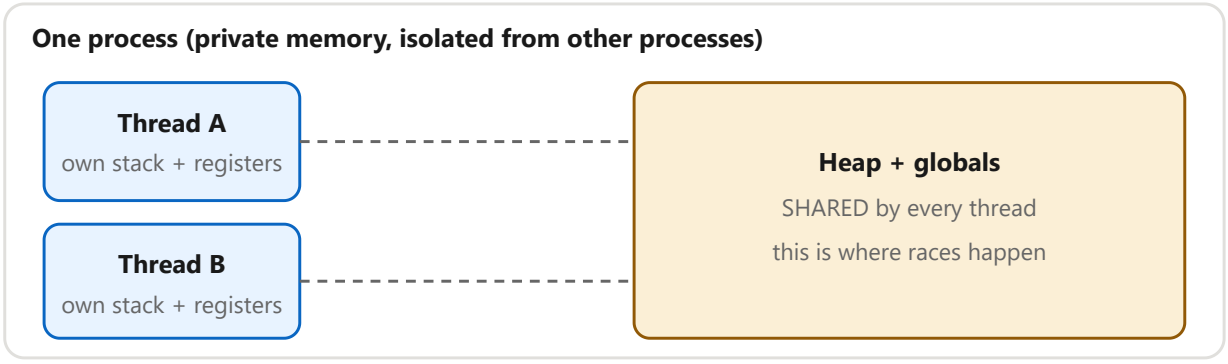
Threads: The Mental Model and Your First Thread

What a thread is, concurrency versus parallelism, the cost of a thread, and how to start, pass arguments to, join and detach one safely. Assumes Function Pointers and Functors and Lambdas.

Why threads? The mental model



Threads share memory; that sharing is the source of both the speed and the bugs.

A **thread** is an independent flow of execution. Several threads make progress at once, like several cooks in one kitchen.

TERM	MEANING
Concurrency	tasks *in progress* at overlapping times (can be interleaved on one core)
Parallelism	tasks running at the *same instant* on different cores

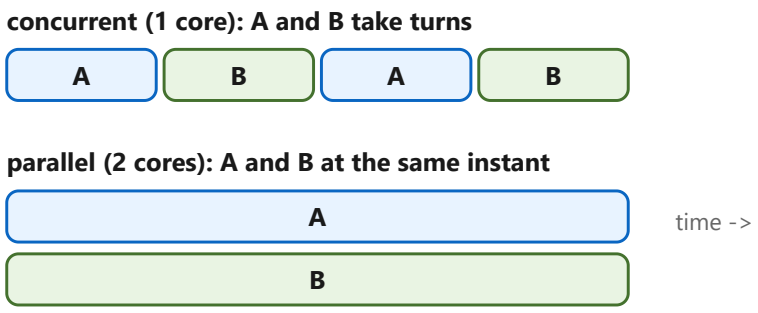
Why: speed (use all cores), responsiveness (keep a UI or network handler alive), hiding I/O waits. **Cost:** about 1 MB stack each, startup time, context switches. Use threads only where the gain is measurable.

> THE WHOLE DIFFICULTY Threads share memory, and the OS can switch between them **between any two instructions**.

```cpp
// how many hardware threads (cores x hyperthreads) this machine has, e.g.
// 8
std::thread::hardware_concurrency();
```

Concurrency, parallelism and the cost of a thread

Two words are often mixed up. **Concurrency** means several tasks are in progress at overlapping times, even if one core just switches between them. **Parallelism** means they really run at the same instant on different cores. Threads give you both, and the operating system decides which one you get.



Concurrent is about structure, parallel is about hardware.

	COST OF A THREAD	WHY IT MATTERS
memory	about 1 MB of stack reserved, plus kernel bookkeeping	thousands of threads exhaust memory
start-up	tens of microseconds to create and schedule	a thread per tiny task is slower than the task
switching	the OS saves and restores registers and caches go cold	more threads than cores means waiting in line

```cpp
// how many hardware threads this machine has, e.g. 8
std::thread::hardware_concurrency();
```

> **RULE OF THUMB** For CPU-bound work use about one thread per hardware thread. For work that mostly waits (network, disk) more threads help, because they sleep instead of computing.

Your first thread

```cpp
#include <iostream>
#include <thread>

void hello(int id) {
    std::cout << "worker " << id << " running\n";
}

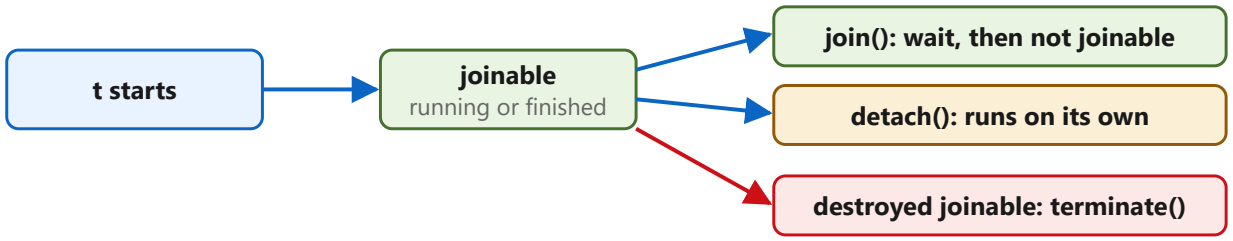
int main() {
    // starts running hello(1) IMMEDIATELY, on a new thread
    std::thread t(hello, 1);
    // main keeps going at the same time
    std::cout << "main continues\n";
    // wait here until t has finished
    t.join();
}
```

The **order is not fixed**. Run it repeatedly and you may see either:

```
main continues            worker 1 running
worker 1 running    or    main continues
```

A test that passed once proves almost nothing.

Arguments, ownership and detach bugs



Every std::thread must be joined or detached before it is destroyed.

- **Always `join()` or `detach()` before the `std::thread` is destroyed**, otherwise `std::terminate`. A detached thread must not touch anything that may be destroyed.
- **Arguments are copied** into the thread. Use `std::ref` to pass a reference.
- Threads are **move-only**: `std::thread b = std::move(a);`.
- An exception escaping the thread function calls `std::terminate`. Catch inside, or pass it out with a `future`.

```
void addOne(int& n) { ++n; }

int x = 0;
// std::ref: pass x by reference (without it this will not even compile)
std::thread t1(addOne, std::ref(x));
t1.join();                                  // x is now 1

std::thread t2([] { std::cout << "a lambda works too\n"; });
t2.join();

struct Worker { void run(int n) { /* ... */ } } w;
// member function: pointer to member, the object, then the arguments
std::thread t3(&Worker::run, &w, 5);
t3.join();

// pause the current thread
std::this_thread::sleep_for(std::chrono::milliseconds(100));
// identify the current thread
std::this_thread::get_id();
```

CLASSIC DETACH BUG

```
void f() {
    int local = 0;
    // captures local BY REFERENCE
    std::thread t([&] { ++local; });
    t.detach();
// f returns, local is destroyed, the thread still writes to it
}
```

Capture by value (or `join()`) if the thread can outlive the variable.

CONVERTED TOO LATE

```
void f(int i, const std::string& s);

void oops() {
    char buffer[1024];
    // copies the char* POINTER, not the text
    std::thread t(f, 3, buffer);
    // oops() may return before the thread builds the std::string
    t.detach();
// fix: std::thread t(f, 3, std::string(buffer));
}
```

The conversion to `std::string` happens later, inside the new thread. Convert on the calling side.

Join on every path

If anything before `join()` can throw, the thread stays joinable and the program terminates. Use RAII:

```
class ThreadGuard {
    std::thread& t_;
public:
    explicit ThreadGuard(std::thread& t) : t_(t) {}
    // joins however the scope is left
    ~ThreadGuard() { if (t_.joinable()) t_.join(); }
    ThreadGuard(const ThreadGuard&) = delete;
    ThreadGuard& operator=(const ThreadGuard&) = delete;
};

void f() {
    std::thread t(work);
    // joins when f exits, even by an exception
    ThreadGuard g(t);
    mayThrow();
}
```

Why no implicit join or detach? A join hides long waits; a detach leaves a thread using dead locals. If a class owns a `std::thread`, declare it **last** among the members.

> **C++20: STD::JTHREAD** Joins automatically in its destructor and supports `std::stop_token` cancellation.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Why threads? The mental model	C++ Concurrency in Action	1.1 What is concurrency?	25 to 30
	C++ Concurrency in Action	1.2.3 When not to use concurrency	32 to 33
	C++ Concurrency in Action	3.3.2 to 3.3.3 Protecting rarely updated data (reader-writer lock); recursive locking	91 to 95
Your first thread	Effective Modern C++	Item 37, Make std::threads unjoinable on all paths	268 to 276
	C++ Concurrency in Action	2.1 Basic thread management (launching, joining, detaching, waiting in exceptional circumstances, running in the background)	40 to 47
	C++ Concurrency in Action	2.2 Passing arguments to a thread function	47 to 50
	C++ Concurrency in Action	2.3 Transferring ownership of a thread; 2.5 Identifying threads	50 to 53 and 57 to 59